

① Timer

↳ It counts till FINAL_VALUE then gives signal TICK.

```
always @ (posedge clk) begin
    if (counter == Finalvalue)
        tick <= 1;
        counter <= 0;
    else
        tick <= 0;
        counter <= counter + 1;
```

↳ you can add reset logic as well

input:	clk	output: tick
	Finalvalue	
	reset	

reg: counter

② PWM Module

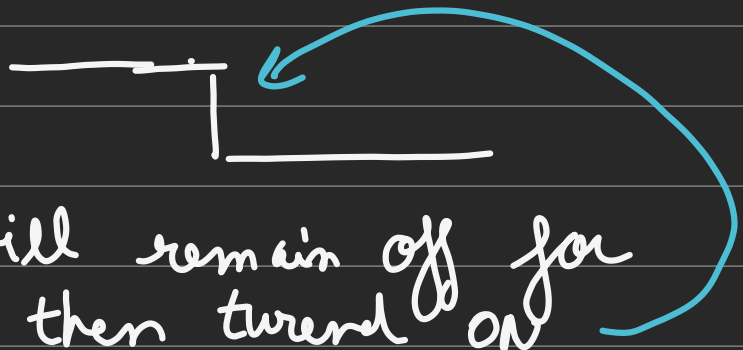
↳ It listens for the tick from timer.

↳ for every tick.

- Add 1 to a counter

- checks if

 counter < Duty → LED ON
 else → OFF



always @ (posedge clk) begin.

 if (tick) begin.

 if (counter = $2^N - 1$)
 counter = 0;

 else

 counter = counter + 1;

 end

end

assign pwm_out = (counter < duty);

input : duty
clk

output: pwm_out

③ RGB Driver

↳ It has 3 LEDs (R, G, B)

↳ Each one gets its own duty & runs independently.

pwm improved RED (.duty (red - duty),
· pwm_out (red - LED));

———— GREEN (.duty (green - duty),
· pwm_out (green - LED));

———— BLUE (.duty (blue - duty),
· pwm_out (blue - LED));

input : Red-duty

output : Red-LED

|

|

② RGB application

- ↳ pick a color with "color-selector".
- ↳ set brightness with "color-intensity".
- ↳ press "change-color".

```
always @ (posedge clk) begin
```

```
  if (change_color) begin
```

```
    use (color_selector)
```

```
    2'b00: R_led <= color_intensity
```

```
    2'b01: G_led <= color_intensity
```

```
    2'b10: B_led <= color_intensity
```

